



DESKTOP

Formation - Partie 2 - .NET

PLAN DE COURS

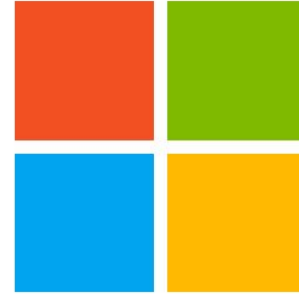
Chapitre 1 : Introduction à C#

Chapitre 2 : Présentation de .NET

1. Introduction à C#

Le langage C#

- Commercialisé par Microsoft en 2002
- Langage Orienté objet
- Simple et polyvalent
- Fortement typé
- Fourni avec le framework .NET



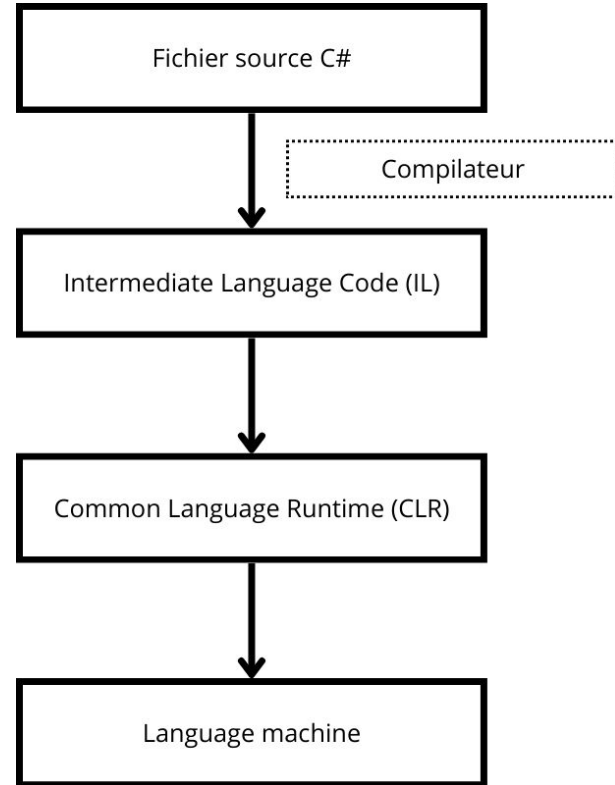
Avantages de C#

- Gestion automatique de la mémoire (**garbage collector**)
- **Interopérabilité**, permettant aux développeurs d'utiliser des API écrites en différents langages sur le framework .NET
- Possibilités de **développement variés** : applications web, mobiles, desktop, ou encore jeux vidéos



Le compilateur C#

- Génération du **code intermédiaire** pouvant être lue par le CLR mais non exécutable par la machine (devient .exe / .dll)
- Compilation dynamique par le **CLR** en code machine spécifique au système d'exploitation de la machine



Environnement d'exécution

- <https://www.programiz.com/csharp-programming/online-compiler/>
- Ce site permet la compilation et l'exécution des programmes

Structure de base d'un programme



```
1  using System; // Importation du namespace System, qui contient des classes de base.
2
3  class Program // Déclaration d'une classe nommée Program.
4  {
5      static void Main(string[] args) // Méthode Main, point d'entrée du programme.
6      {
7          Console.WriteLine("Bonjour, monde !"); // Affiche le texte dans la console.
8      }
9  }
```

Déclaration des variables



```
1  int age = 25;           // Déclaration d'une variable entière nommée 'age' avec une valeur de 25.
2  double taille = 1.75;   // Déclaration d'une variable flottante 'taille' initialisée à 1.75.
3  string nom = "Alice";   // Déclaration d'une variable de type chaîne 'nom' initialisée à "Alice".
4  bool estEtudiant = true; // Déclaration d'une variable booléenne 'estEtudiant' initialisée à vrai.
```

Instructions conditionnelles



```
1  if (age >= 18) // Vérifie si 'age' est supérieur ou égal à 18.
2  {
3      Console.WriteLine("Vous êtes majeur."); // Si vrai, affiche ce message.
4  }
5  else
6  {
7      Console.WriteLine("Vous êtes mineur."); // Sinon, affiche ce message.
8  }
```



```
1  switch (age) // Utilise l'âge pour déterminer quel bloc de code exécuter.
2  {
3      case 18: // Si 'age' est égal à 18.
4          Console.WriteLine("Vous venez d'avoir 18 ans."); // Affiche ce message.
5          break; // Termine le bloc switch.
6      default: // Si aucune des cases précédentes ne correspond.
7          Console.WriteLine("Vous avez un âge."); // Affiche ce message.
8          break; // Termine le bloc switch.
9  }
```

Exercices

Ex 1.1

- Créez un programme C# qui affiche le message suivant dans la console :
 - Bienvenue au cours de programmation en C# !

Ex 1.2

- Déclarez les variables suivantes :
 - int nombre1 avec une valeur de 10.
 - int nombre2 avec une valeur de 20.
- Ensuite, créez une troisième variable int somme qui stocke la somme de nombre1 et nombre2.
- Affichez le résultat avec le message : La somme de 10 et 20 est : 30

Ex 1.3

- Créez un programme qui demande à l'utilisateur de saisir sa note (entre 0 et 20, pas de vérification sur la saisie).
- Le programme doit ensuite évaluer la performance de l'utilisateur selon la note saisie :
 - Si la note est entre 0 et 9, affichez : "Insuffisant."
 - Si la note est entre 10 et 15, affichez : "Assez bien."
 - Si la note est entre 16 et 20, affichez : "Bien."
- Une version avec **switch** et une avec **if**

Boucle for



```
1  for (int i = 0; i < 10; i++) // Boucle qui commence à 0 et s'arrête avant 10.  
2  {  
3      Console.WriteLine(i); // Affiche la valeur de 'i' à chaque itération.  
4  }
```

Boucle while



```
1  while (age < 30) // Continue tant que 'age' est inférieur à 30.  
2  {  
3      age++; // Incrémente 'age' de 1 à chaque itération.  
4  }  
5
```

Création et appel de méthode



```
1  static void SePresenter()  
2  {  
3      Console.WriteLine("Je m'appelle Romain");  
4  }  
5  
6  // Lorsque la méthode est static, pas besoin de créer une instance de la classe  
7  // SePresenter(); pour appeler la methode  
8  
9  static int calculAddition(int a, int b)  
10 {  
11     return a + b;  
12 }  
13  
14 int result = calculAddition(2, 3);  
15 Console.WriteLine(result); // affiche 5 sur la console
```

Exemple des méthodes avec une classe

```
1  using System; // nécessaire pour la classe Console
2
3  class Program
4  {
5      static void Main(string[] args)
6      {
7          SePresenter(); // Appelle la méthode de présentation
8
9          int resultat = CalculAddition(5, 3); // Appelle la méthode d'addition
10         Console.WriteLine("La somme est : " + resultat); // Affiche le résultat de l'addition
11     }
12
13     static void SePresenter()
14     {
15         Console.WriteLine("Je m'appelle Romain");
16     }
17
18     static int CalculAddition(int a, int b)
19     {
20         return a + b; // Renvoie la somme de 'a' et 'b'
21     }
22 }
```

Exemple avec getter / setter

```
1 using System; // Nécessaire pour la classe Console
2
3 class Program
4 {
5     static void Main(string[] args)
6     {
7         // Création d'une instance de la classe Personne
8         Personne personne = new Personne();
9         personne.Nom = "Romain"; // Utilisation du setter pour définir le nom
10        personne.Age = 30; // Utilisation du setter pour définir l'âge
11
12        personne.SePresenter(); // Appelle la méthode pour se présenter
13
14        int resultat = CalculAddition(5, 3); // Appelle la méthode d'addition
15        Console.WriteLine("La somme est : " + resultat); // Affiche le résultat de l'addition
16    }
17
18    static int CalculAddition(int a, int b)
19    {
20        return a + b; // Renvoie la somme de 'a' et 'b'
21    }
22 }
```

```
1 using System;
2
3 class Personne
4 {
5     // Propriété pour le nom
6     public string Nom { get; set; }
7
8     // Propriété pour l'âge
9     public int Age { get; set; }
10
11    // Méthode pour se présenter
12    public void SePresenter()
13    {
14        Console.WriteLine($"Je m'appelle {Nom} et j'ai {Age} ans.");
15    }
16 }
```

Exercices

Ex 2.1

- Créez un programme qui demande à l'utilisateur de saisir un nombre.
- Utilisez une boucle for pour afficher la table de multiplication de ce nombre (jusqu'à 10).
- Par exemple, si l'utilisateur entre "3", le programme doit afficher :
 - $3 \times 1 = 3$
 - $3 \times 2 = 6$
 - ...
 - $3 \times 10 = 30$

Ex 2.2

- Créez un programme qui utilise une boucle while pour calculer la somme des nombres pairs entre 1 et un nombre saisi par l'utilisateur.
- Par exemple, si l'utilisateur entre "10", le programme doit afficher la somme des nombres pairs entre 1 et 10 :
 - $2 + 4 + 6 + 8 + 10 = 30$

Ex 2.3

- Créez une classe Livre avec les propriétés titre (string) et nbPages (int)
- Ajoutez getters et setters pour chaque propriété (pour nbPages, le nombre de pages doit être supérieur ou égal à 1, sinon affichez une erreur).
- Ajoutez une méthode AfficherInfos() qui affiche le titre et le nombre de pages du livre dans la console.
- Dans le Main, créez une instance de Livre, affectez-lui un titre et un nombre de pages, puis appelez la méthode AfficherInformations().

Les tableaux

```
1  class Program
2  {
3      static void Main(string[] args)
4      {
5          // Création d'un tableau d'entiers avec une taille fixe
6          int[] nombres = new int[5]; // Tableau de 5 éléments
7          // Si l'on connaît les nombres, on peut faire int[] nombres = {10, 20, 30, 40, 50};
8
9          int compteur = 0; // Compteur pour garder une trace du nombre d'éléments ajoutés
10
11         // Ajout de nombres au tableau
12         nombres[compteur++] = 10; // Ajoute 10 et incrémente le compteur
13         nombres[compteur++] = 20; // Ajoute 20
14         nombres[compteur++] = 30; // Ajoute 30
15
16         // Suppression d'un nombre (simulée par la mise à zéro)
17         nombres[1] = 0; // Simule la suppression de 20 en le remplaçant par 0
18
19         // Ajout d'un nouveau nombre
20         if (compteur < nombres.Length)
21         {
22             nombres[compteur++] = 40; // Ajoute 40 si le tableau n'est pas plein
23         }
24     }
25 }
26 }
```

Exercices

Ex 3.1

- Créez un programme qui demande à l'utilisateur de saisir 5 notes (entiers) et les stocke dans un tableau.
- Ensuite, le programme doit calculer et afficher :
 - La liste des notes saisies.
 - La moyenne de ces notes.

Les collections

Une collection est une structure qui permet de regrouper et de gérer plusieurs éléments.

Il existe plusieurs types de collections, notamment les listes, les dictionnaires, les ensembles...

Les listes

Une liste est une collection dynamique qui peut contenir un nombre variable d'éléments.

La classe `List<T>` est utilisée pour créer des listes de type générique.

- **Dynamique** : Les listes peuvent grandir ou rétrécir selon les besoins : ajout ou suppression d'éléments à tout moment
- **Types de données** : Les listes peuvent contenir des types de données variés (chaînes de caractères, entiers, objets...)
- **Par index** : Les éléments d'une liste peuvent être accédés par leur index, comme dans un tableau.

Les listes

```
1 using System.Collections.Generic; // Nécessaire pour utiliser List<T>
2
3 class Program
4 {
5     static void Main(string[] args)
6     {
7         // Création d'une liste de chaînes de caractères
8         List<string> noms = new List<string>();
9
10        // Ajout de noms à la liste
11        noms.Add("Alice");
12        noms.Add("Bob");
13        noms.Add("Charlie");
14
15        // Suppression d'un nom
16        noms.Remove("Bob"); // Supprime "Bob" de la liste
17
18        // Ajout d'un nouveau nom
19        noms.Add("David");
20    }
21 }
```

Exercices

Ex 4.1

- Créez un programme qui demande à l'utilisateur de saisir 5 nombres entiers et les ajoute à une liste.
- Ensuite, affichez uniquement les nombres pairs de la liste.

Les dictionnaires

Un dictionnaire est une collection qui associe des clés à des valeurs.

La classe `Dictionary<TKey, TValue>` est utilisée pour créer des dictionnaires de type générique.

- **Dynamique** : Les dictionnaires peuvent grandir ou rétrécir selon les besoins : ajout ou suppression d'éléments à tout moment.
- **Types de données** : Les dictionnaires peuvent contenir des types de données variés, où les clés et les valeurs peuvent être de différents types (par exemple, chaînes de caractères pour les clés et entiers pour les valeurs).
- **Par clé** : Les éléments d'un dictionnaire peuvent être accédés par leur clé, ce qui permet de retrouver rapidement une valeur associée.

Les dictionnaires

```
1 using System;
2 using System.Collections.Generic; // Nécessaire pour utiliser Dictionary
3
4 class Program
5 {
6     static void Main(string[] args)
7     {
8         // Un dictionnaire est une collection qui associe des clés à des valeurs.
9         Dictionary<string, int> ages = new Dictionary<string, int>();
10
11         // Les dictionnaires peuvent grandir selon les besoins : ajout d'éléments à tout moment.
12         ages.Add("Alice", 30);
13         ages.Add("Bob", 25);
14         ages.Add("Charlie", 35);
15
16         // Dans cet exemple, les clés sont des chaînes de caractères et les valeurs sont des entiers.
17
18         // Les éléments d'un dictionnaire peuvent être accédés par leur clé.
19         Console.WriteLine("Âge d'Alice : " + ages["Alice"]);
20         Console.WriteLine("Âge de Bob : " + ages["Bob"]);
21
22         // Les dictionnaires permettent également de supprimer des éléments.
23         ages.Remove("Bob"); // Supprime Bob du dictionnaire
24
25         Console.WriteLine("\nAprès suppression de Bob :");
26
27         // exemple pour afficher les éléments du dictionnaire
28         foreach (var entry in ages)
29         {
30             Console.WriteLine($"{entry.Key} : {entry.Value} ans");
31         }
32     }
33 }
34
```

Exercices

Ex 5.1

- Créez un programme qui utilise un dictionnaire pour stocker le prix de 3 produits. Chaque produit est identifié par un nom (clé) et un prix (valeur).
- Ensuite, demandez à l'utilisateur de saisir le nom d'un produit pour afficher son prix.
- Si le produit n'existe pas, affichez un message d'erreur.

C# : gestion des exceptions

```
1 using System; // Nécessaire pour la classe Console
2
3 class Program
4 {
5     static void Main(string[] args)
6     {
7         try
8         {
9             Console.WriteLine("Entrez un nombre entier :");
10            int nombre = Convert.ToInt32(Console.ReadLine());
11            // Valeur attendue : 100 ou -100 ou 0 ou autre
12            // Peut lever une exception (Format d'entrée invalide par exemple : FormatException)
13
14            // Simuler une division par zéro
15            int resultat = 100 / nombre; // Peut lever une exception DivideByZeroException
16            Console.WriteLine($"Le résultat de 100 divisé par {nombre} est {resultat}.");
17        }
18        catch (FormatException ex) // Gestion d'une exception de format
19        {
20            Console.WriteLine("Erreur : Vous devez entrer un nombre valide.");
21        }
22        catch (DivideByZeroException ex) // Gestion d'une exception de division par zéro
23        {
24            Console.WriteLine("Erreur : Division par zéro n'est pas autorisée.");
25        }
26        catch (Exception ex) // Gestion d'autres exceptions non spécifiées
27        {
28            Console.WriteLine($"Une erreur s'est produite : {ex.Message}");
29        }
30        finally // Optionnel, exécuté même si une exception est levée
31        {
32            Console.WriteLine("Fin du traitement."); // Ce bloc sera toujours exécuté
33        }
34    }
35 }
```

Exercices

Ex 6.1

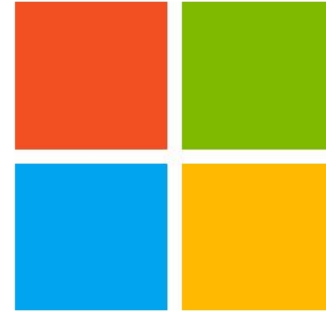
- Créez un programme qui demande à l'utilisateur de saisir un nombre entier sous forme de chaîne de caractères.
- Le programme doit tenter de convertir cette chaîne en entier et afficher le résultat. Utilisez les exceptions pour gérer les cas suivants :
 - Si la saisie ne peut pas être convertie en entier, affichez un message d'erreur.
 - Si la conversion réussit, affichez le nombre.
 - Utilisez “finally” pour afficher un message indiquant que la tentative de conversion est terminée.

TP 2 : Validation des acquis

2. Présentation de .NET

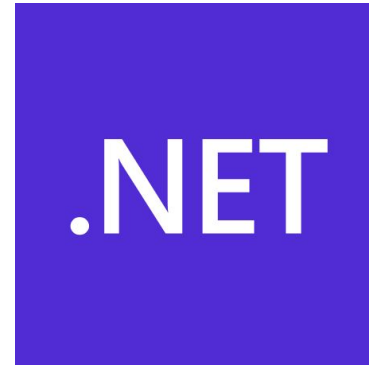
La plateforme .NET

- Développé par **microsoft**
- Moderniser le développement d'applications windows à l'origine
- Utilisé pour créer des applications variées (bureautique, web, mobile, IoT...)
- Modèle supportant plusieurs langages (**C#**, VB, Managed C++)
- Multiplateforme : windows, macOS, Linux



Les composants clés de .NET

- Un environnement d'exécution (**Common Language Runtime**) qui gère notamment l'exécution du code, la gestion de mémoire, la sécurité.
- Il permet d'exécuter des programmes développés dans différents langages
- Le **Framework Class Library (FCL)**, une bibliothèque de classes incluant des fonctionnalités pour la gestion des données, les interfaces graphiques, et autres ...



Evolution de .NET : .NET Framework

- Lancement en 2002 de .NET Framework pour répondre aux besoins des applications windows
- .NET framework 4.8 depuis 2019
- Enfermé dans l'écosystème windows et peu adapté aux technologies modernes (cloud, DevOps par exemple)

Evolution de .NET : .NET Core

- Lancement en 2016 de .NET Core pour répondre à l'évolution des besoins, en particulier le développement multiplateforme.
- Open source, supporte les microservices, adapté aux environnements conteneurisés
- Dernière version en date .NET Core 3.X depuis 2019
- Ce projet n'est plus mis à jour depuis l'unification en 2020

Evolution de .NET : .NET 5 à 10

- En 2020, création d'une plateforme unique : .NET 5
- Un framework incluant les capacités de .NET core avec les bibliothèques de .NET framework.
- Toujours maintenu, avec de nouvelles fonctionnalités (.NET 6 en 2021, .NET 7 en 2022 et .NET 10 en 2025)

Evolution de .NET : résumé

